

HandVerify: A Comparative Study of Deep Learning Architectures for Writer Verification

Lorenzo Arcioni^{1*}† and Enrico Giordani^{1†}

^{1*}Department of Computer Science, Sapienza University of Rome, Via Salaria 113, Rome, 00198, Italy.

*Corresponding author(s). E-mail(s): arcioni.1885377@studenti.uniroma1.it;

Contributing authors: giordani.2143412@studenti.uniroma1.it;

†These authors contributed equally to this work.

Abstract

Writer verification is a critical biometric task with applications in forensic document analysis, security systems, and authentication. This paper presents HandVerify, a modular framework for offline handwriting verification using deep Siamese neural networks. We systematically evaluate three training objectives—Binary Cross-Entropy (BCE) classification, Contrastive Loss and Triplet Loss—across six ImageNet-pretrained CNN backbones drawn from the ResNet, EfficientNet and MobileNetV3 families, on two benchmark datasets: IAM (English) and RIMES (French). We train and evaluate all 72 (backbone, loss, dataset-pairing) combinations over two within-dataset and two cross-dataset pairings, reporting EER, AUC, decidability and genuine acceptance rate at fixed low false-acceptance rates. The two metric-learning objectives consistently outperform BCE, with Contrastive the most consistent across backbones and splits (paired Wilcoxon signed-rank, $p < 10^{-6}$). The best same-dataset configuration reaches an EER of 3.1% (IAM→IAM) and the best cross-dataset configuration 8.2% (RIMES→IAM), while genuine acceptance degrades sharply at FAR = 0.1%—showing that a low EER does not imply usable operation under strict false-acceptance constraints. Each configuration was trained once, with a single random seed and a single writer-disjoint split; the reported figures are therefore point estimates. All code and experimental protocols are publicly available.

Keywords: Handwriting Verification, Siamese Networks, Contrastive Learning, Triplet Loss, Computer Vision, Deep Learning, Biometric Authentication, Transfer Learning

1 Experimental Setup

1.1 Datasets

We evaluate our framework on two standard handwriting verification datasets. Due to the modular design, the framework supports any dataset organized with writer subdirectories containing handwriting samples. For this study, we use datasets with varying numbers of writers and samples per writer to assess generalization across different data scales.

Images are preprocessed to grayscale and resized to 448×448 pixels, balancing computational efficiency with fine detail preservation. This resolution is sufficient to capture stroke characteristics while remaining manageable for modern GPUs.

1.2 Data Preprocessing

Both IAM and RIMES are normalized to a fixed 448×448 size while preserving handwriting characteristics. For each image, an Otsu-binarized mask is derived from the grayscale version and used as an auxiliary guide to locate the handwritten region and compute a tight crop via axial projections. For RIMES, the resulting coordinates are applied to the original *grayscale* image, so the mask serves purely as a guide; for IAM, they are applied to the *binarized* image itself, which is the one resized and saved.

IAM. The handwritten region is extracted from each form by detecting the pre-printed guide lines (with a fixed-region fallback), then cleaned, tightly cropped, and resized to a square. Each form yields the full image plus four patches from a 2×2 grid, for up to five images per form.

RIMES. Text lines are cropped, resized to a common height, concatenated per writer into a strip, cut into 448-pixel segments, and stacked in groups of 5 to form 448×448 batches. Batches of the same writer are further combined into 2×2 mosaics.

Images with too little ink are discarded. All formulations, parameter values, and fallback conditions are given in Appendix A.1. These choices are designed around the final dataset, in particular to avoid having too few positive/negative pairs or triplets, which could make training more difficult.

1.3 Dataset Statistics Analysis

After preprocessing, we analyze the distribution of images per author across the two resulting datasets, as this directly informs downstream design choices such as batching strategy and writer-level splitting. Table 1 summarizes the key statistics computed on the preprocessed IAM and RIMES datasets.

Table 1 Summary statistics of images per author for the preprocessed IAM and RIMES datasets.

Dataset	Authors	Tot. Images	Mean/Author	Median/Author	Std/Author	Min/Max
IAM	657	1539	2.34	1.0	3.03	1 / 59
RIMES	1500	5046	3.36	3.0	1.27	1 / 10

IAM comprises 657 authors contributing a total of 1539 processed images, with a mean of 2.34 images per author but a median of only 1.0, indicating a strongly right-skewed distribution; this is confirmed by the high standard deviation (3.03) relative to the mean, and by the presence of an extreme outlier author with 59 images, more than an order of magnitude above the median. RIMES, in contrast, comprises 1500 authors and 5046 total processed images (batches), with mean and median again fairly close (3.36 and 3.0 respectively) and a moderate standard deviation (1.27), with a maximum of 10 images per author. The corresponding per-author histograms and boxplot comparison, which visually confirm this asymmetry (IAM dominated by a large spike at 1–2 images per author with a long sparse tail, RIMES broader and more centrally distributed), are reported in Appendix A.2.

This asymmetry has practical implications for the training pipeline: since IAM images already correspond to full, variable-length handwritten samples (one 448×448 image per crop), no further batching is applied, and the heavy-tailed per-author distribution is instead handled through a writer-level (rather than image-level) train/validation/test split, preventing data leakage across sets. For RIMES, the "images" counted here correspond to the stacked 448×448 batches produced by the line-grouping procedure (Appendix A.1); the broader spread around 3 batches per author, compared to IAM, reflects the larger and more variable number of lines each RIMES writer contributed, combined with the fixed batch size $N_{\text{batch}} = 5$.

1.4 Experimental Protocol

1.4.1 Train/Validation/Test Splitting and Dataset Pairings

Each candidate backbone/loss configuration is evaluated on four train→evaluation dataset pairings (IAM→IAM, IAM→RIMES, RIMES→IAM, RIMES→RIMES), for $6(\text{backbones}) \times 3(\text{losses}) \times 4(\text{pairings}) = 72$ trained models in total.

For the two **within-dataset** pairings (IAM→IAM, RIMES→RIMES), the source dataset is writer-disjointly split into three subsets: 70% training, 10% validation (`val_size` = 0.1, used for model selection / early-stopping monitoring), and 20% test (`test_size` = 0.2, held out and used only for the final comprehensive evaluation), with a fixed random seed of 42.

For the two **cross-dataset** pairings (IAM→RIMES, RIMES→IAM), the model is trained on the *entire* source dataset (all of its writers), while the target dataset is itself split writer-disjointly into a 10% validation subset (monitoring/early stopping) and a 90% test subset, the latter used for the final evaluation reported throughout this paper. This setting assesses the ability of a model trained on one handwriting dataset to generalize to a different handwriting domain and data-collection protocol.

Exact writer and pair counts per split are given in Table ?? (Section ??); these counts are confirmed identical across all backbone/loss combinations within a given split, since the writer-level split depends only on the fixed random seed and dataset identity, not on backbone or loss.

For all training configurations, negative samples are resampled every epoch, since the number of possible negative pairs is substantially larger than the number of positive pairs. Resampling therefore allows the model

to observe a wider variety of negative examples throughout training, rather than repeatedly using the same subset of negative pairs.

This choice is also motivated by the intended real-world application: in forensic handwriting and signature examination, cases are not submitted to an examiner at random, but precisely because the authenticity of a signature or document is already in question—an instance of what is known in the forensic statistics literature as an *ascertainment* (or *selection*) bias effect on casework composition [1]¹. As a result, the population of samples an examiner actually evaluates is skewed toward disputed, and often forged, material relative to the general population of signatures. Exposing the model to a wide and varied set of negative examples during training is therefore consistent with this real-world prior: a system intended to support forgery detection should be robust across a broad range of negative (forged) cases, rather than a narrow, fixed subset.

No resampling is applied to the validation or test sets, which remain fixed throughout each experiment. Each model is trained from scratch for every architecture/loss/dataset-pairing combination, with early stopping monitored using the corresponding validation set.

1.4.2 Hyperparameters

All 72 configurations of the main sweep share a single uniform hyperparameter configuration: batch size 16, 3 frozen backbone encoder layers, dropout 0.2, and margins 1.0 (Contrastive) / 0.5 (Triplet), matching the values introduced in Section ???. A complete list of representative hyperparameters, including values not otherwise discussed in the main text (maximum epochs, embedding dimension, data-loading workers, etc), is given in Appendix A.3.

Acknowledgements. The authors would like to thank Kaggle for providing access to GPU computational resources, which enabled the training and evaluation of the models presented in this work.

Declarations

Funding. Not applicable.

Conflict of interest. The authors declare no competing interests.

Ethics approval. Not applicable.

Data availability. The datasets used in this study are publicly available at the dataset links: [RIMES](#) and [IAM](#)

Code availability. The HandVerify framework is implemented in PyTorch and it is publicly available on [GitHub](#) under the MIT License.

Author contributions. Both authors contributed equally to this work. L.A. designed the framework architecture and conducted the training and testing experiments. E.G. implemented the application design, deployed the application, verified its correct functionality. Both authors wrote and reviewed the manuscript.

Appendix A Implementation Details

A.1 Preprocessing Pipeline Details

This appendix provides the full mathematical formulation of the preprocessing pipeline summarized in Section 1.

A.1.1 Binarization as an Auxiliary Mask

Each image is first converted to grayscale and an Otsu-thresholded binary version is derived from it. Otsu’s method determines the optimal threshold t^* by maximizing inter-class variance:

$$t^* = \arg \max_t \sigma_B^2(t) = \arg \max_t \omega_0(t) \omega_1(t) [\mu_0(t) - \mu_1(t)]^2 \quad (\text{A1})$$

where $\omega_0(t)$ and $\omega_1(t)$ are the background and foreground class probabilities, and $\mu_0(t)$, $\mu_1(t)$ their respective means. In the resulting mask, text pixels are 0 (black) and background pixels are 255 (white). The mask is used to: (i) detect IAM guide lines, (ii) locate left-margin black columns and top/bottom black rows, and (iii) compute tight-crop bounding boxes. For RIMES, the coordinates derived from the mask are applied to the grayscale image, which is what is cropped, resized, and saved. For IAM, the coordinates are applied to the binary image, which is what is cropped, resized, and saved.

¹The cited work discusses this effect in the context of firearms examiner casework; we adopt it here as a conceptually analogous argument, as no equivalent study (at the best of our knowledge) specifically quantifying casework composition exists for forensic handwriting/signature examination.

A.1.2 Tight Cropping

To eliminate excessive whitespace while preserving text content, we compute tight crop coordinates using axial projections on the binary mask. For a binary image I of size $h \times w$, we invert it ($I_{\text{inv}} = 255 - I$) and compute row and column projections:

$$S_{\text{row}}(i) = \sum_{j=1}^w \mathbb{I}[I_{\text{inv}}(i, j) > 0], \quad S_{\text{col}}(j) = \sum_{i=1}^h \mathbb{I}[I_{\text{inv}}(i, j) > 0] \quad (\text{A2})$$

Adaptive thresholds determine content-bearing rows and columns, based on a minimum ink-density ratio ρ :

$$\tau_{\text{row}} = \max(10, \lfloor \rho \cdot w \rfloor), \quad \tau_{\text{col}} = \max(10, \lfloor \rho \cdot h \rfloor) \quad (\text{A3})$$

We use $\rho = 0.018$ for RIMES lines and a slightly stricter $\rho = 0.02$ for IAM, to compensate for residual guide-line artifacts left after region extraction. The bounding box is

$$y_1 = \min\{i : S_{\text{row}}(i) > \tau_{\text{row}}\}, \quad y_2 = \max\{i : S_{\text{row}}(i) > \tau_{\text{row}}\} \quad (\text{A4})$$

$$x_1 = \min\{j : S_{\text{col}}(j) > \tau_{\text{col}}\}, \quad x_2 = \max\{j : S_{\text{col}}(j) > \tau_{\text{col}}\} \quad (\text{A5})$$

If no row or no column exceeds its threshold, the image is discarded. A 3-pixel internal trim, applied only along the row axis, removes residual guide lines: $y_1 \leftarrow \min(y_1 + 3, y_2 - 1)$ and $y_2 \leftarrow \max(y_2 - 3, y_1 + 1)$, while x_1, x_2 are left untouched. This is followed by 5-pixel padding on *all four* sides, clipped to the image bounds, to prevent text from touching borders. The resulting coordinates (y_1, y_2, x_1, x_2) are used to crop the image.

A.1.3 Aspect-Preserving Resize to Square

For IAM, images are normalized to 448×448 pixels by first centering them on a white (255) square canvas of side $s = \max(h, w)$ with offsets

$$\text{offset}_y = \lfloor (s - h)/2 \rfloor, \quad \text{offset}_x = \lfloor (s - w)/2 \rfloor \quad (\text{A6})$$

The canvas is then resized to 448×448 using area interpolation, preserving the original aspect ratio without distortion.

A.1.4 Ink-Based Quality Filtering

We define the ink ratio of an image as the fraction of pixels with intensity below 250. Images whose ink ratio falls below a threshold are discarded (and stored in a separate “rejected” folder for inspection). The thresholds are: 0.010 for full IAM images, 0.010 for IAM patches, 0.015 for RIMES 448×448 batches (which contain white padding), and 0.020 for RIMES mosaics.

A.1.5 IAM Dataset Preprocessing

IAM forms contain pre-printed guide lines that must be excluded. We detect horizontal lines using morphological opening (erosion followed by dilation, 2 iterations) on the inverted binary mask, with a kernel of width $k_w = \max(30, \lfloor w/15 \rfloor)$:

$$\text{Opening}(I, K) = \text{Dilate}(\text{Erode}(I, K), K) \quad (\text{A7})$$

Valid lines satisfy $w_c > 0.3w$, $h_c < 20$, and area > 100 pixels. Detected lines are sorted top-to-bottom and the handwritten region is selected as follows:

- **Three or more lines detected:** the region is extracted between the second and third detected lines,

$$y_{\text{start}} = y_{\text{bottom}}^{(2)} + 8, \quad y_{\text{end}} = y_{\text{top}}^{(3)} - 8 \quad (\text{A8})$$

- **Exactly two lines detected:** the region is extracted between them,

$$y_{\text{start}} = y_{\text{bottom}}^{(1)} + 8, \quad y_{\text{end}} = y_{\text{top}}^{(2)} - 8 \quad (\text{A9})$$

- **Fewer than two lines detected:** a secondary detection pass estimates guide-line positions from a smoothed row-density profile. The row-wise ink density is smoothed with a uniform filter (window size 5), and rows exceeding the 95th percentile of the smoothed profile are grouped into contiguous segments (gap tolerance of 5 pixels). If this pass yields three or more segments, or exactly two, the same rules above are re-applied to the segments; otherwise the pipeline uses the fixed fallback.

- **Fallback:** if no reliable lines are found by either pass,

$$y_{\text{start}} = \lfloor 0.22h \rfloor, \quad y_{\text{end}} = \lfloor 0.85h \rfloor \quad (\text{A10})$$

Two sanity checks guard against degenerate cases. First, if $y_{\text{start}} \geq y_{\text{end}}$ after any of the branches above, the pipeline reverts to the fixed fallback $[0.22h, 0.85h]$. Second, y_{start} and y_{end} are clipped to the valid image range, $y_{\text{start}} \leftarrow \max(0, y_{\text{start}})$ and $y_{\text{end}} \leftarrow \min(h, y_{\text{end}})$. As a final safeguard, if the extracted region is empty or smaller than 30×50 pixels (height $\times$ width), extraction reverts to the fixed fallback region. Region selection is performed on the binary mask, and the same rows are extracted from both the grayscale and binary images to keep them spatially aligned.

Left black column removal. Let r_j be the fraction of black pixels in column j of the binary region. We remove all columns to the left of

$$j^* = \min\{j : r_k \leq 0.02 \quad \forall k \in [j, j + 19]\} \quad (\text{A11})$$

i.e., the start of the first run of at least 20 consecutive “clean” columns. This handles both sharp black bands and gradual decay at scan borders. If no such run exists, no column is removed.

Full image and patches. After column removal, tight cropping ($\rho = 0.02$, Appendix A.1.2) is applied and the cropped binary region is resized to a square 448×448 image (the full sample). If its ink ratio is below the threshold, the whole form is discarded. Otherwise, the cropped region is also divided into a 2×2 grid of patches (the last row/column absorbing any remainder); each patch is independently centered on a square canvas, resized to 448×448 , and kept only if it passes the ink-ratio filter. Each form therefore yields up to five images. Figure A1 illustrates the pipeline.

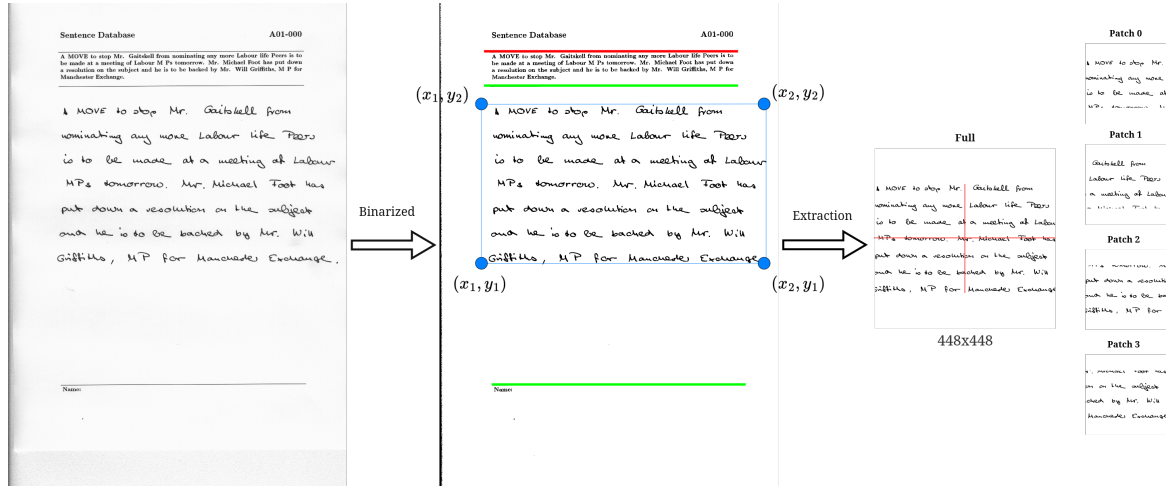


Fig. A1 Preprocessing of an IAM form: the grayscale image (left) is binarized (center) to detect guide lines (red/green, detected by erosion/dilation) and the handwritten text region (blue box); this region is then extracted, cropped, and resized to 448×448 pixels (right). The cropped region is also split into a 2×2 grid of patches, each independently resized to 448×448 pixels, so that each form yields up to five images.

A.1.6 RIMES Dataset Preprocessing

RIMES consists of individual text lines. Lines are grouped by writer, identified by the split name and the writer index in the filename (e.g., `eval2011-0`). The goal is to turn the lines of each writer into 448×448 images with a text density comparable to the IAM samples. Two constants define the layout: each image contains $N_{\text{batch}} = 5$ rows of text, so every row has height $H = \lfloor 448/N_{\text{batch}} \rfloor = 89$ pixels. The processing has three stages: (i) each line is cleaned and resized independently, (ii) the lines of a writer are assembled into 448×448 batches, and (iii) batches are combined into mosaics.

Stage 1: line-level processing.

Each line is converted to grayscale, and an auxiliary Otsu mask is derived from it. The mask is used only to compute crop coordinates, which are then applied to the grayscale line; only the grayscale line is resized and saved. Each line undergoes the following steps:

1. **Vertical trim:** dark bands at the top and bottom of the scan are removed. We discard the rows in which more than 50% of the pixels are black, keeping the range between the first and the last row that are not.

2. **Tight cropping:** the whitespace around the text is removed using the projection-based algorithm with $\rho = 0.018$ (Appendix A.1.2).
3. **Splitting:** lines wider than 896 pixels are split at the midpoint into two segments, to avoid excessively long lines after resizing.
4. **Height normalization:** every line is resized to height $H = 89$ pixels, preserving the aspect ratio, so that its width becomes $w' = \text{round}(w \cdot H/h)$. No horizontal padding is applied.

After this stage, every line has the same height and a variable width.

Stage 2: batch construction (per writer).

The lines of a writer are concatenated horizontally into a single long strip, with a 15-pixel white gap between consecutive lines so that they are not read as one. The strip is then cut every 448 pixels into slices of size 89×448 (the last one is right-padded with white). Note that a slice does not correspond to an original line: it may contain the end of one line and the beginning of the next. Finally, $N_{\text{batch}} = 5$ consecutive slices are stacked vertically, giving an image of size 445×448 , and 3 white rows are appended at the bottom to obtain a 448×448 batch. Batches with too little ink (typically the last one, which is often almost empty) are discarded according to the ink-ratio filter.

Stage 3: mosaics.

To obtain more samples per writer, the surviving batches are combined into 2×2 mosaics: each batch is downsampled to 224×224 and four batches, taken in generation order, are tiled into a 448×448 image. For example, a writer with 9 batches yields three mosaics: batches 1–4, batches 5–8, and a final one made of batch 9 plus three batches already used, with the tile positions randomly shuffled. If a writer has fewer than four batches, the missing tiles are left white. Mosaics with too little ink are discarded. Both batches and mosaics are saved as outputs. Figure A2 illustrates stages 1 and 2.

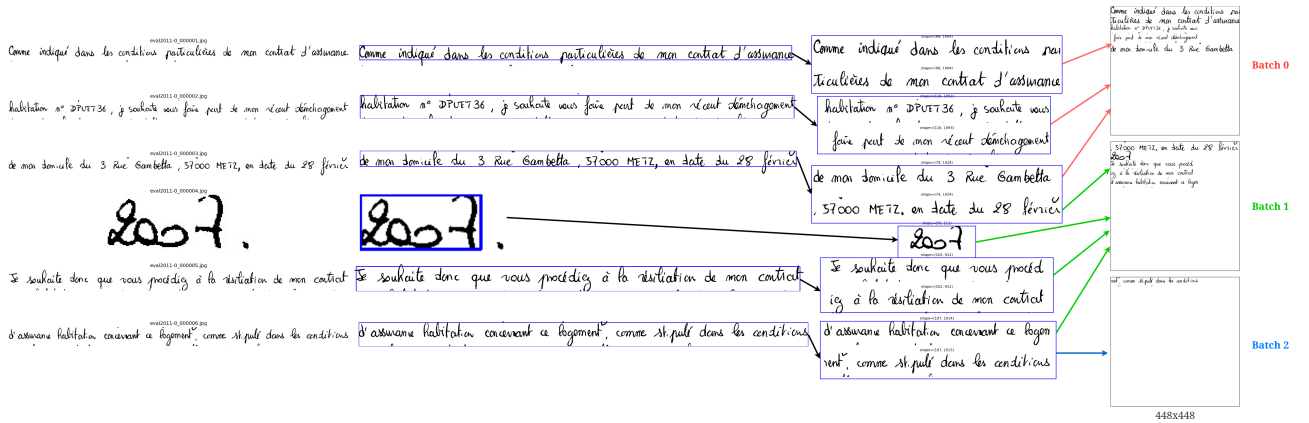


Fig. A2 Preprocessing of RIMES text lines for a given writer. Each original line (left, e.g. eval2011-0.000001.jpg) is binarized via Otsu thresholding and tight-cropped (blue bounding boxes, center) using the projection-based algorithm; the corresponding grayscale crop is then extracted and resized to a common height (right of center). The lines of the writer are concatenated into a single strip, cut into slices of width 448 pixels, and 5 slices are stacked vertically into a 448×448 batch (right), with unused space padded in white. Slices do not necessarily align with the original lines. Batches are subsequently combined into 2×2 mosaics (not shown).

A.2 Dataset Statistics: Distribution Figures

Figure A3 shows the per-author image count distributions for both datasets after preprocessing, complementing the summary statistics of Table 1. The IAM histogram (top) is dominated by a large spike at 1–2 images per author, with a long, sparse tail extending up to the outlier author with 59 images. The RIMES histogram is comparatively broader and shifted toward higher counts, peaking around 3 batches per author and extending up to 10, reflecting a more even but still right-skewed spread. The boxplot comparison (bottom) further highlights this asymmetry: IAM’s interquartile range is narrow and close to zero, with the extreme outlier and long upper whisker revealing a much heavier tail, while RIMES shows a wider, more centrally distributed spread with a shorter relative tail.

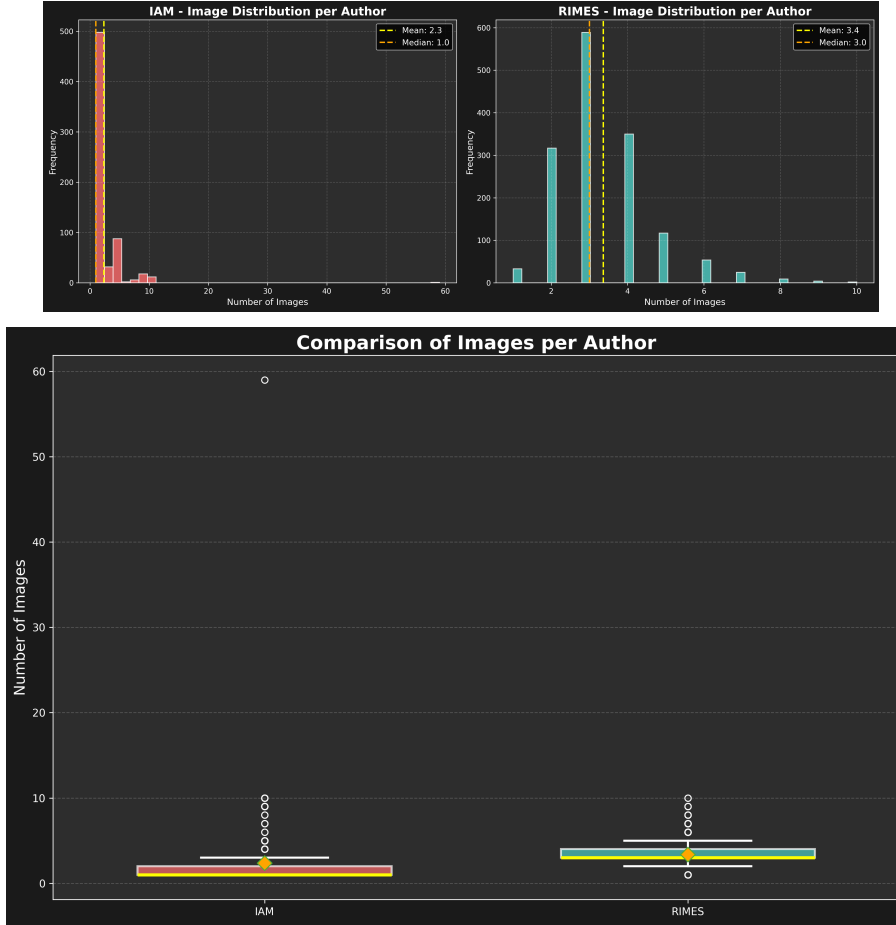


Fig. A3 Distribution of images per author for the preprocessed IAM and RIMES datasets. **Top:** per-dataset histograms with mean (yellow) and median (orange) overlaid, showing the strong right skew of IAM compared to the broader, more centrally distributed RIMES spread. **Bottom:** side-by-side boxplot comparison, highlighting the extreme outlier author in IAM (59 images) against the wider but shorter-tailed RIMES distribution (max 10 images).

A.3 Hyperparameter Configuration

Table A1 summarizes the hyperparameter configuration used across the 72-model sweep, including values already discussed in the main text (Section 1) and additional training-setup details not otherwise reported, such as maximum epochs, embedding dimension, and data-loading workers. Loss-specific margins follow Section ??: $m = 1.0$ for Contrastive and $m = 0.5$ for Triplet.

Table A1 Hyperparameter configuration shared across the 72 model configurations. Loss-specific settings are indicated where applicable.

Hyperparameter	Configuration
Batch size	16
Maximum epochs	50
Early stopping	Patience = 5
Image size	448 × 448 pixels
Embedding dimension	32
Margin	1.0 (Contrastive), 0.5 (Triplet)
Positive ratio	0.5
Dropout	0.2
Frozen encoder layers	3
Negative resampling	Every epoch (all losses)
Data-loading workers	4

A.4 Training Dynamics

Figure A4 reports training and validation loss curves for the ResNet-18 backbone across all 12 loss×split combinations. Across nearly all configurations, training loss decreases monotonically, consistent with stable

optimization; however, validation-loss behavior differs substantially by loss function and split. Under BCE, validation loss tracks training loss relatively closely on the same-dataset splits (IAM→IAM, RIMES→RIMES), while the cross-dataset splits show an early divergence between the two curves (e.g., IAM→RIMES), consistent with the same-vs-cross-dataset generalization gap discussed in Section ?? . Under Contrastive and, more markedly, under Triplet Loss, validation loss is considerably noisier and frequently non-monotonic, with pronounced oscillations and, in several cases (e.g., Triplet on IAM→IAM and IAM→RIMES), no clear downward trend despite steadily decreasing training loss – indicative of a widening train–validation gap rather than smooth joint convergence. Training run lengths also vary considerably across configurations, from as few as 6 epochs (Triplet, RIMES→RIMES) to over 40 (BCE, IAM→IAM), reflecting the early-stopping criterion (patience 5, Appendix A.3) being triggered at different points depending on the stability of the validation signal for each configuration.

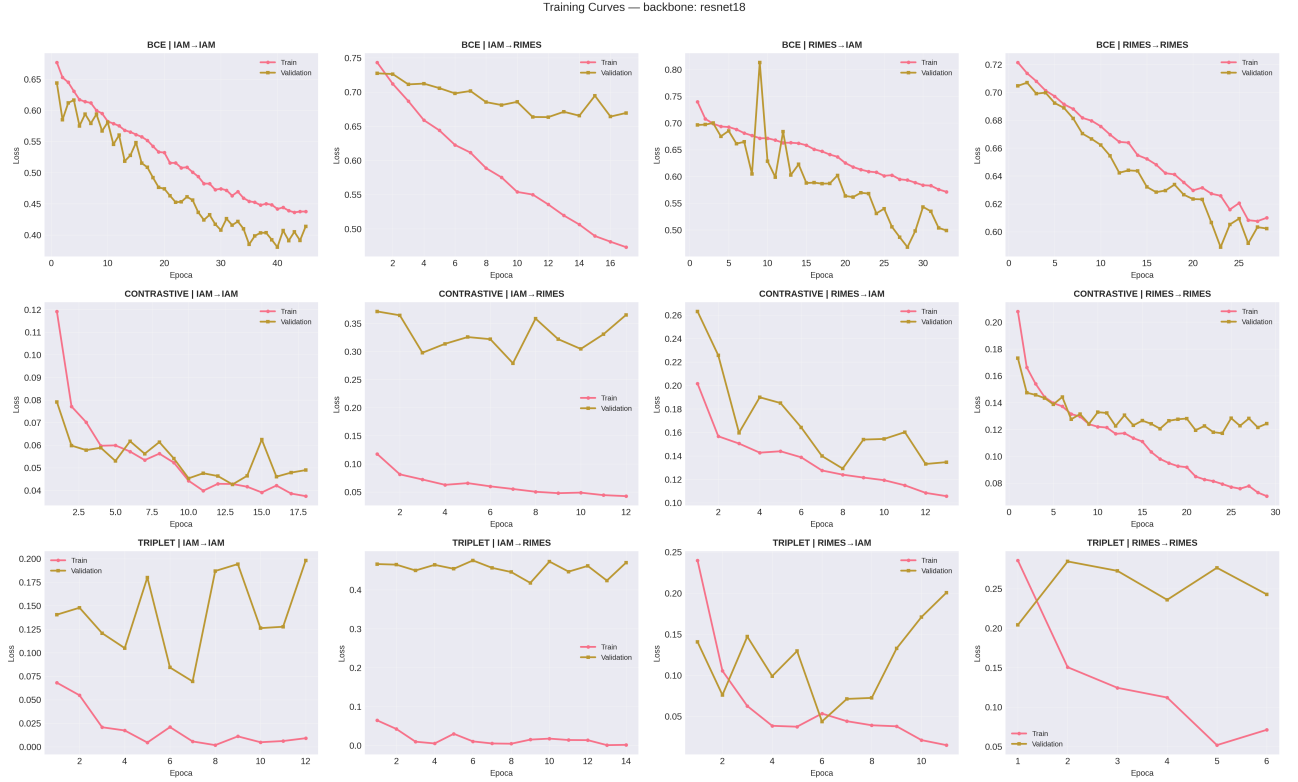


Fig. A4 Training and validation loss curves, ResNet-18, all 12 loss×split combinations.

References

- [1] Rosenblum, M., Garrett, B.L., Scurich, N., *et al.*: Incorrect statistical reasoning in Guyl et al. leads to biased claims about strength of forensic evidence. Proceedings of the National Academy of Sciences (2024) <https://doi.org/10.1073/pnas.2315431121>